

LTMB12 – Hospital Management SQL Practice

– Question, Query, Output

MySQL 8.0 | hospital_db | 7 tables (patients, doctors, appointments, billing, treatments, medicines, prescriptions)

Source Data (as inserted)

patients

patient_id	patient_name	gender	age	city
1	Rahul	Male	35	Chennai
2	Priya	Female	28	Bangalore
3	Arun	Male	45	Hyderabad
4	Sneha	Female	31	Mumbai
5	Kiran	Male	50	Delhi
6	Ramesh Kumar	Male	42	Chennai
7	Lakshmi Devi	Female	36	Coimbatore
8	Suresh Babu	Male	29	Madurai
9	Anitha Rao	Female	55	Bangalore
10	Vijay Sharma	Male	61	Mumbai
11	Deepa Nair	Female	48	Kochi
12	Karthik Raj	Male	34	Chennai
13	Meena Patel	Female	40	Ahmedabad
14	Rohit Singh	Male	27	Delhi
15	Pooja Gupta	Female	31	Hyderabad
16	Naveen Kumar	Male	45	Pune
17	Shalini Verma	Female	52	Lucknow

patient_id	patient_name	gender	age	city
18	Ajay Das	Male	38	Kolkata
19	Neha Reddy	Female	24	Hyderabad
20	Manoj Jain	Male	58	Jaipur

doctors

doctor_id	doctor_name	specialization	consultation_fee
101	Dr. Kumar	Cardiology	1000
102	Dr. Mehta	Neurology	1500
103	Dr. Sharma	Orthopedics	1200
104	Dr. Reddy	Dermatology	800
105	Dr. Anjali Iyer	Pediatrics	900
106	Dr. Rajesh Gupta	ENT	850
107	Dr. Vivek Menon	General Medicine	700
108	Dr. Sunita Shah	Gynecology	1300
109	Dr. Akash Verma	Cardiology	1200
110	Dr. Harish Rao	Neurology	1800
111	Dr. Kavitha Devi	Orthopedics	1400
112	Dr. Prakash Nair	Pulmonology	1100

appointments

appointment_id	patient_id	doctor_id	appointment_date	status
1001	1	101	2025-05-01	Completed
1002	2	102	2025-05-03	Completed
1003	3	101	2025-05-05	Pending
1004	4	103	2025-05-06	Completed
1005	5	104	2025-05-08	Cancelled
1006	6	105	2025-05-10	Completed
1007	7	106	2025-05-11	Completed

appointment_id	patient_id	doctor_id	appointment_date	status
1008	8	107	2025-05-11	Completed
1009	9	108	2025-05-12	Pending
1010	10	109	2025-05-12	Completed
1011	11	110	2025-05-13	Completed
1012	12	111	2025-05-13	Completed
1013	13	112	2025-05-14	Cancelled
1014	14	105	2025-05-14	Completed
1015	15	106	2025-05-15	Completed
1016	16	107	2025-05-16	Pending
1017	17	108	2025-05-17	Completed
1018	18	109	2025-05-17	Completed
1019	19	110	2025-05-18	Completed
1020	20	111	2025-05-18	Pending
1021	1	112	2025-05-19	Completed
1022	2	105	2025-05-20	Completed
1023	3	106	2025-05-20	Completed
1024	4	107	2025-05-21	Completed
1025	5	108	2025-05-21	Cancelled

billing

bill_id	appointment_id	bill_amount	payment_status
1	1001	1000	Paid
2	1002	1500	Paid
3	1003	1000	Pending
4	1004	1200	Paid
5	1005	0	Cancelled
6	1006	900	Paid
7	1007	850	Paid

bill_id	appointment_id	bill_amount	payment_status
8	1008	700	Paid
9	1009	1300	Pending
10	1010	1200	Paid
11	1011	1800	Paid
12	1012	1400	Paid
13	1013	0	Cancelled
14	1014	900	Paid
15	1015	850	Paid
16	1016	700	Pending
17	1017	1300	Paid
18	1018	1200	Paid
19	1019	1800	Paid
20	1020	1400	Pending
21	1021	1100	Paid
22	1022	900	Paid
23	1023	850	Paid
24	1024	700	Paid
25	1025	0	Cancelled

treatments

treatment_id	appointment_id	treatment_name	treatment_cost
1	1001	ECG	500
2	1002	Brain Scan	2500
3	1004	Knee X-Ray	1200
4	1006	Vaccination	300
5	1007	Ear Cleaning	400
6	1008	General Checkup	250
7	1010	ECG	500

treatment_id	appointment_id	treatment_name	treatment_cost
8	1011	MRI Scan	3500
9	1012	Bone Scan	1800
10	1014	Vaccination	300
11	1015	ENT Checkup	500
12	1017	Pregnancy Scan	2200
13	1018	Stress Test	900
14	1019	Neurology Test	4000
15	1021	Pulmonary Function Test	1600

medicines

medicine_id	medicine_name	medicine_price
1	Paracetamol	20
2	Amoxicillin	120
3	Aspirin	50
4	Metformin	180
5	Atorvastatin	250
6	Omeprazole	90
7	Cetirizine	40
8	Vitamin D	150

prescriptions

prescription_id	appointment_id	medicine_id	quantity
1	1001	1	10
2	1001	3	5
3	1002	2	7
4	1004	5	15
5	1006	7	10
6	1007	6	5

prescription_id	appointment_id	medicine_id	quantity
7	1008	1	10
8	1010	4	20
9	1011	5	30
10	1012	3	15
11	1014	8	10
12	1015	7	5
13	1017	2	10
14	1018	1	20
15	1019	5	15

(An *appointment_audit* table also exists in the schema but is empty – reserved for trigger practice – so it has no data to list here.)

Alias convention: tables are aliased *a, b, c, d, e...* alphabetically in the order they appear from **FROM** through the last **JOIN** – never the table's own first letter. Each **JOIN** sits on its own line, with **ON** on the line right below it. Independent subqueries/derived tables restart their own scope at *a* .

All outputs below were computed by actually loading the exact dataset from the script and running each query, so every row and number is verified against the real data.

Part 1 – Easy Level (2 marks each)

E1. Inner Join – Completed Appointments

Question: Display Patient Name, Doctor Name, Appointment Date for all appointments with status = 'Completed'.

Query:

```
-- a=appointments, b=patients, c=doctors
SELECT b.patient_name AS Patient_Name, c.doctor_name AS Doctor_Name, a.ap
FROM appointments a
INNER JOIN patients b
```

```

ON b.patient_id = a.patient_id
INNER JOIN doctors c
ON c.doctor_id = a.doctor_id
WHERE a.status = 'Completed'
ORDER BY a.appointment_date, b.patient_name;

```

Output:

Patient_Name	Doctor_Name	Appointment_Date
Rahul	Dr. Kumar	2025-05-01
Priya	Dr. Mehta	2025-05-03
Sneha	Dr. Sharma	2025-05-06
Ramesh Kumar	Dr. Anjali Iyer	2025-05-10
Lakshmi Devi	Dr. Rajesh Gupta	2025-05-11
Suresh Babu	Dr. Vivek Menon	2025-05-11
Vijay Sharma	Dr. Akash Verma	2025-05-12
Deepa Nair	Dr. Harish Rao	2025-05-13
Karthik Raj	Dr. Kavitha Devi	2025-05-13
Rohit Singh	Dr. Anjali Iyer	2025-05-14
Pooja Gupta	Dr. Rajesh Gupta	2025-05-15
Ajay Das	Dr. Akash Verma	2025-05-17
Shalini Verma	Dr. Sunita Shah	2025-05-17
Neha Reddy	Dr. Harish Rao	2025-05-18
Rahul	Dr. Prakash Nair	2025-05-19
Arun	Dr. Rajesh Gupta	2025-05-20
Priya	Dr. Anjali Iyer	2025-05-20
Sneha	Dr. Vivek Menon	2025-05-21

E2. Left Join – Appointments with Billing

Question: Display Appointment ID, Appointment Date, Bill Amount for all appointments, even if billing information is missing.

Query:

```
-- a=appointments, b=billing
SELECT a.appointment_id AS Appointment_ID, a.appointment_date AS Appointm
FROM appointments a
LEFT JOIN billing b
ON b.appointment_id = a.appointment_id
ORDER BY a.appointment_id;
```

Output:

Appointment_ID	Appointment_Date	Bill_Amount
1001	2025-05-01	1000
1002	2025-05-03	1500
1003	2025-05-05	1000
1004	2025-05-06	1200
1005	2025-05-08	0
1006	2025-05-10	900
1007	2025-05-11	850
1008	2025-05-11	700
1009	2025-05-12	1300
1010	2025-05-12	1200
1011	2025-05-13	1800
1012	2025-05-13	1400
1013	2025-05-14	0
1014	2025-05-14	900
1015	2025-05-15	850
1016	2025-05-16	700
1017	2025-05-17	1300
1018	2025-05-17	1200
1019	2025-05-18	1800
1020	2025-05-18	1400
1021	2025-05-19	1100

Appointment_ID	Appointment_Date	Bill_Amount
1022	2025-05-20	900
1023	2025-05-20	850
1024	2025-05-21	700
1025	2025-05-21	0

(Every appointment in this dataset happens to have a billing row, so the LEFT JOIN result matches an INNER JOIN here – but LEFT JOIN is the correct choice since the requirement is "show all appointments even if billing is missing.")

E3. Simple Subquery – Doctors Above Average Fee

Question: Display Doctor ID, Doctor Name, Consultation Fee for doctors whose fee is greater than the average consultation fee of all doctors.

Query:

```
SELECT doctor_id AS Doctor_ID, doctor_name AS Doctor_Name, consultation_f
FROM doctors
WHERE consultation_fee > (SELECT AVG(consultation_fee) FROM doctors)
ORDER BY consultation_fee DESC;
```

Output:

Doctor_ID	Doctor_Name	Consultation_Fee
110	Dr. Harish Rao	1800
102	Dr. Mehta	1500
111	Dr. Kavitha Devi	1400
108	Dr. Sunita Shah	1300
103	Dr. Sharma	1200
109	Dr. Akash Verma	1200

(Average fee across 12 doctors = 1112.50.)

E4. Join with Aggregate – Total Appointments per Doctor

Question: Display Doctor Name and Total Appointments for every doctor.

Query:

```
-- a=doctors, b=appointments
SELECT a.doctor_name AS Doctor_Name, COUNT(b.appointment_id) AS Total_App
FROM doctors a
LEFT JOIN appointments b
ON b.doctor_id = a.doctor_id
GROUP BY a.doctor_id, a.doctor_name
ORDER BY Total_Appointments DESC, a.doctor_name;
```

Output:

Doctor_Name	Total_Appointments
Dr. Anjali Iyer	3
Dr. Rajesh Gupta	3
Dr. Sunita Shah	3
Dr. Vivek Menon	3
Dr. Akash Verma	2
Dr. Harish Rao	2
Dr. Kavitha Devi	2
Dr. Kumar	2
Dr. Prakash Nair	2
Dr. Mehta	1
Dr. Reddy	1
Dr. Sharma	1

E5. Join with Aggregate + HAVING – Doctors with More Than 2 Appointments

Question: Display Doctor Name and Total Appointments for doctors who handled more than 2 appointments.

Query:

```
-- a=doctors, b=appointments
SELECT a.doctor_name AS Doctor_Name, COUNT(b.appointment_id) AS Total_Appointments
FROM doctors a
INNER JOIN appointments b
ON b.doctor_id = a.doctor_id
GROUP BY a.doctor_id, a.doctor_name
HAVING COUNT(b.appointment_id) > 2
ORDER BY Total_Appointments DESC, a.doctor_name;
```

Output:

Doctor_Name	Total_Appointments
Dr. Anjali Iyer	3
Dr. Rajesh Gupta	3
Dr. Sunita Shah	3
Dr. Vivek Menon	3

Part 2 – Medium Level (4 marks each)

M1. Join + Subquery – Patients Who Consulted Cardiology Doctors

Question: Display Patient Name, Doctor Name, Appointment Date for patients who consulted Cardiology doctors. Use a subquery to first identify all Cardiology doctors.

Query:

```
-- a=appointments, b=patients, c=doctors
SELECT b.patient_name AS Patient_Name, c.doctor_name AS Doctor_Name, a.appointment_date AS Appointment_Date
FROM appointments a
INNER JOIN patients b
ON b.patient_id = a.patient_id
INNER JOIN doctors c
ON c.doctor_id = a.doctor_id
WHERE c.specialty = 'Cardiology';
```

```

INNER JOIN doctors c
ON c.doctor_id = a.doctor_id
WHERE c.doctor_id IN (
    SELECT doctor_id
    FROM doctors
    WHERE specialization = 'Cardiology'
)
ORDER BY a.appointment_date, b.patient_name;

```

Output:

Patient_Name	Doctor_Name	Appointment_Date
Rahul	Dr. Kumar	2025-05-01
Arun	Dr. Kumar	2025-05-05
Vijay Sharma	Dr. Akash Verma	2025-05-12
Ajay Das	Dr. Akash Verma	2025-05-17

M2. Join + Aggregate + Subquery – Doctors Above Average Revenue

Question: Display Doctor Name and Total Revenue Generated, only for doctors whose total revenue is above the average doctor revenue.

Query:

```

-- outer: a=doctors, b=appointments, c=billing
-- derived table (own scope): a=doctors, b=appointments, c=billing
SELECT a.doctor_name AS Doctor_Name, SUM(c.bill_amount) AS Total_Revenue_
FROM doctors a
INNER JOIN appointments b
ON b.doctor_id = a.doctor_id
INNER JOIN billing c
ON c.appointment_id = b.appointment_id
GROUP BY a.doctor_id, a.doctor_name
HAVING SUM(c.bill_amount) > (
    SELECT AVG(Doctor_revenue)
    FROM (
        SELECT SUM(c.bill_amount) AS Doctor_revenue
        FROM doctors a

```

```

        INNER JOIN appointments b
        ON b.doctor_id = a.doctor_id
        INNER JOIN billing c
        ON c.appointment_id = b.appointment_id
        GROUP BY a.doctor_id
    ) AS Doctor_totals
)
ORDER BY Total_Revenue_Generated DESC;

```

Output:

Doctor_Name	Total_Revenue_Generated
Dr. Harish Rao	3600
Dr. Kavitha Devi	2800
Dr. Anjali Iyer	2700
Dr. Sunita Shah	2600
Dr. Rajesh Gupta	2550
Dr. Akash Verma	2400
Dr. Vivek Menon	2100

(Average doctor revenue across all 12 doctors = 2045.83.)

M3. Window Function (ROW_NUMBER) – Most Recent Appointment per Patient

Question: Display Patient Name, Appointment Date, Doctor Name – the latest appointment of each patient – using ROW_NUMBER().

Query:

```

-- inner: a=appointments, b=patients, c=doctors
SELECT Patient_Name, Appointment_Date, Doctor_Name
FROM (
    SELECT b.patient_name AS Patient_Name, a.appointment_date AS Appointment_Date,
           c.doctor_name AS Doctor_Name,
           ROW_NUMBER() OVER (
               PARTITION BY a.patient_id
               ORDER BY a.appointment_date DESC, a.appointment_id DESC
           ) AS Row_Num
    FROM appointments a
    JOIN patients b
    JOIN doctors c
    ON b.patient_id = a.patient_id
    ON c.doctor_id = a.doctor_id
)
WHERE Row_Num = 1;

```

```

    ) AS Rn
FROM appointments a
INNER JOIN patients b
ON b.patient_id = a.patient_id
INNER JOIN doctors c
ON c.doctor_id = a.doctor_id
) AS Ranked_appointments
WHERE Rn = 1
ORDER BY Patient_Name;

```

Output:

Patient_Name	Appointment_Date	Doctor_Name
Ajay Das	2025-05-17	Dr. Akash Verma
Anitha Rao	2025-05-12	Dr. Sunita Shah
Arun	2025-05-20	Dr. Rajesh Gupta
Deepa Nair	2025-05-13	Dr. Harish Rao
Karthik Raj	2025-05-13	Dr. Kavitha Devi
Kiran	2025-05-21	Dr. Sunita Shah
Lakshmi Devi	2025-05-11	Dr. Rajesh Gupta
Manoj Jain	2025-05-18	Dr. Kavitha Devi
Meena Patel	2025-05-14	Dr. Prakash Nair
Naveen Kumar	2025-05-16	Dr. Vivek Menon
Neha Reddy	2025-05-18	Dr. Harish Rao
Pooja Gupta	2025-05-15	Dr. Rajesh Gupta
Priya	2025-05-20	Dr. Anjali Iyer
Rahul	2025-05-19	Dr. Prakash Nair
Ramesh Kumar	2025-05-10	Dr. Anjali Iyer
Rohit Singh	2025-05-14	Dr. Anjali Iyer
Shalini Verma	2025-05-17	Dr. Sunita Shah
Sneha	2025-05-21	Dr. Vivek Menon
Suresh Babu	2025-05-11	Dr. Vivek Menon
Vijay Sharma	2025-05-12	Dr. Akash Verma

M4. Window Function (RANK) – Rank Doctors by Total Revenue

Question: Display Doctor Name, Total Revenue, Revenue Rank using RANK().

Query:

```
-- derived table: a=doctors, b=appointments, c=billing
SELECT Doctor_Name, Total_Revenue,
       RANK() OVER (ORDER BY Total_Revenue DESC) AS Revenue_Rank
FROM (
  SELECT a.doctor_id, a.doctor_name AS Doctor_Name,
         SUM(c.bill_amount) AS Total_Revenue
  FROM doctors a
  INNER JOIN appointments b
  ON b.doctor_id = a.doctor_id
  INNER JOIN billing c
  ON c.appointment_id = b.appointment_id
  GROUP BY a.doctor_id, a.doctor_name
) AS Doctor_revenue
ORDER BY Revenue_Rank, Doctor_Name;
```

Output:

Doctor_Name	Total_Revenue	Revenue_Rank
Dr. Harish Rao	3600	1
Dr. Kavitha Devi	2800	2
Dr. Anjali Iyer	2700	3
Dr. Sunita Shah	2600	4
Dr. Rajesh Gupta	2550	5
Dr. Akash Verma	2400	6
Dr. Vivek Menon	2100	7
Dr. Kumar	2000	8
Dr. Mehta	1500	9
Dr. Sharma	1200	10
Dr. Prakash Nair	1100	11

Doctor_Name	Total_Revenue	Revenue_Rank
Dr. Reddy	0	12

M5. Complex Join + Aggregate + Window – Top Doctor per Specialization

Question: Display Specialization, Doctor Name, Total Revenue – the top revenue-generating doctor from each specialization.

Query:

```
-- derived table: a=doctors, b=appointments, c=billing
SELECT Specialization, Doctor_Name, Total_Revenue
FROM (
    SELECT a.specialization AS Specialization, a.doctor_name AS Doctor_Name,
           SUM(c.bill_amount) AS Total_Revenue,
           ROW_NUMBER() OVER (
               PARTITION BY a.specialization
               ORDER BY SUM(c.bill_amount) DESC, a.doctor_id
           ) AS Rn
    FROM doctors a
    INNER JOIN appointments b
    ON b.doctor_id = a.doctor_id
    INNER JOIN billing c
    ON c.appointment_id = b.appointment_id
    GROUP BY a.doctor_id, a.doctor_name, a.specialization
) AS Spec_revenue
WHERE Rn = 1
ORDER BY Specialization;
```

Output:

Specialization	Doctor_Name	Total_Revenue
Cardiology	Dr. Akash Verma	2400
Dermatology	Dr. Reddy	0
ENT	Dr. Rajesh Gupta	2550
General Medicine	Dr. Vivek Menon	2100

Specialization	Doctor_Name	Total_Revenue
Gynecology	Dr. Sunita Shah	2600
Neurology	Dr. Harish Rao	3600
Orthopedics	Dr. Kavitha Devi	2800
Pediatrics	Dr. Anjali Iyer	2700
Pulmonology	Dr. Prakash Nair	1100

Part 3 – 20 Scenario-Based Questions (Joins + Window Functions)

S1. Latest Appointment for Every Patient

Question: Display Patient Name, Appointment Date, Doctor Name – the most recent appointment for every patient.

Query:

```
-- inner: a=appointments, b=patients, c=doctors
SELECT Patient_Name, Appointment_Date, Doctor_Name
FROM (
    SELECT b.patient_name AS Patient_Name, a.appointment_date AS Appointment_Date,
           c.doctor_name AS Doctor_Name,
           ROW_NUMBER() OVER (
               PARTITION BY a.patient_id
               ORDER BY a.appointment_date DESC, a.appointment_id DESC
           ) AS Rn
    FROM appointments a
    INNER JOIN patients b
    ON b.patient_id = a.patient_id
    INNER JOIN doctors c
    ON c.doctor_id = a.doctor_id
) AS Ranked_appointments
WHERE Rn = 1
ORDER BY Patient_Name;
```

Output: (identical logic and result to M3 above – see M3 output table.)

S2. Rank Doctors Based on Consultation Fees

Question: Rank all doctors based on consultation fee from highest to lowest: Doctor Name, Specialization, Consultation Fee, Rank.

Query:

```
SELECT doctor_name AS Doctor_Name, specialization AS Specialization, consultation_fee AS Consultation_Fee,
       RANK() OVER (ORDER BY consultation_fee DESC) AS Rank_num
FROM doctors
ORDER BY Rank_num, Doctor_Name;
```

Output:

Doctor_Name	Specialization	Consultation_Fee	Rank_num
Dr. Harish Rao	Neurology	1800	1
Dr. Mehta	Neurology	1500	2
Dr. Kavitha Devi	Orthopedics	1400	3
Dr. Sunita Shah	Gynecology	1300	4
Dr. Akash Verma	Cardiology	1200	5
Dr. Sharma	Orthopedics	1200	5
Dr. Prakash Nair	Pulmonology	1100	7
Dr. Kumar	Cardiology	1000	8
Dr. Anjali Iyer	Pediatrics	900	9
Dr. Rajesh Gupta	ENT	850	10
Dr. Reddy	Dermatology	800	11
Dr. Vivek Menon	General Medicine	700	12

S3. Highest Bill Generated by Each Patient

Question: Display the highest bill amount generated by every patient: Patient Name, Bill Amount, Appointment Date.

Query:

```

-- inner: a=patients, b=appointments, c=billing
SELECT Patient_Name, Bill_Amount, Appointment_Date
FROM (
    SELECT a.patient_name AS Patient_Name, c.bill_amount AS Bill_Amount,
           b.appointment_date AS Appointment_Date,
           ROW_NUMBER() OVER (
               PARTITION BY a.patient_id
               ORDER BY c.bill_amount DESC, b.appointment_id DESC
           ) AS Rn
    FROM patients a
    INNER JOIN appointments b
    ON b.patient_id = a.patient_id
    INNER JOIN billing c
    ON c.appointment_id = b.appointment_id
) AS Ranked_bills
WHERE Rn = 1
ORDER BY Patient_Name;

```

Output:

Patient_Name	Bill_Amount	Appointment_Date
Ajay Das	1200	2025-05-17
Anitha Rao	1300	2025-05-12
Arun	1000	2025-05-05
Deepa Nair	1800	2025-05-13
Karthik Raj	1400	2025-05-13
Kiran	0	2025-05-21
Lakshmi Devi	850	2025-05-11
Manoj Jain	1400	2025-05-18
Meena Patel	0	2025-05-14
Naveen Kumar	700	2025-05-16
Neha Reddy	1800	2025-05-18
Pooja Gupta	850	2025-05-15
Priya	1500	2025-05-03
Rahul	1100	2025-05-19

Patient_Name	Bill_Amount	Appointment_Date
Ramesh Kumar	900	2025-05-10
Rohit Singh	900	2025-05-14
Shalini Verma	1300	2025-05-17
Sneha	1200	2025-05-06
Suresh Babu	700	2025-05-11
Vijay Sharma	1200	2025-05-12

S4. Rank Doctors by Total Revenue

Question: Rank doctors based on total billing revenue: Doctor Name, Total Revenue, Revenue Rank.

Query:

```
-- derived table: a=doctors, b=appointments, c=billing
SELECT Doctor_Name, Total_Revenue,
       RANK() OVER (ORDER BY Total_Revenue DESC) AS Revenue_Rank
FROM (
  SELECT a.doctor_id, a.doctor_name AS Doctor_Name,
         SUM(c.bill_amount) AS Total_Revenue
  FROM doctors a
  INNER JOIN appointments b
  ON b.doctor_id = a.doctor_id
  INNER JOIN billing c
  ON c.appointment_id = b.appointment_id
  GROUP BY a.doctor_id, a.doctor_name
) AS Doctor_revenue
ORDER BY Revenue_Rank, Doctor_Name;
```

Output: (identical to M4 – see M4 output table.)

S5. Most Expensive Treatment Handled by Each Doctor

Question: Display the most expensive treatment performed under each doctor: Doctor Name, Treatment Name, Treatment Cost. One record per doctor.

Query:

```

-- inner: a=doctors, b=appointments, c=treatments
SELECT Doctor_Name, Treatment_Name, Treatment_Cost
FROM (
    SELECT a.doctor_name AS Doctor_Name, c.treatment_name AS Treatment_Na
          c.treatment_cost AS Treatment_Cost,
          ROW_NUMBER() OVER (
              PARTITION BY a.doctor_id
              ORDER BY c.treatment_cost DESC, c.treatment_id DESC
          ) AS Rn
    FROM doctors a
    INNER JOIN appointments b
    ON b.doctor_id = a.doctor_id
    INNER JOIN treatments c
    ON c.appointment_id = b.appointment_id
) AS Ranked_treatments
WHERE Rn = 1
ORDER BY Doctor_Name;

```

Output:

Doctor_Name	Treatment_Name	Treatment_Cost
Dr. Akash Verma	Stress Test	900
Dr. Anjali Iyer	Vaccination	300
Dr. Harish Rao	Neurology Test	4000
Dr. Kavitha Devi	Bone Scan	1800
Dr. Kumar	ECG	500
Dr. Mehta	Brain Scan	2500
Dr. Prakash Nair	Pulmonary Function Test	1600
Dr. Rajesh Gupta	ENT Checkup	500
Dr. Sharma	Knee X-Ray	1200
Dr. Sunita Shah	Pregnancy Scan	2200
Dr. Vivek Menon	General Checkup	250

(Dr. Reddy has no treatment records at all, so is naturally excluded.)

S6. Appointment Numbering for Patients

Question: Assign a sequence number to every appointment of each patient based on appointment date:
Patient Name, Appointment Date, Visit Number.

Query:

```
-- a=appointments, b=patients
SELECT b.patient_name AS Patient_Name, a.appointment_date AS Appointment_
      ROW_NUMBER() OVER (
          PARTITION BY a.patient_id
          ORDER BY a.appointment_date, a.appointment_id
      ) AS Visit_Number
FROM appointments a
INNER JOIN patients b
ON b.patient_id = a.patient_id
ORDER BY b.patient_name, Visit_Number;
```

Output:

Patient_Name	Appointment_Date	Visit_Number
Ajay Das	2025-05-17	1
Anitha Rao	2025-05-12	1
Arun	2025-05-05	1
Arun	2025-05-20	2
Deepa Nair	2025-05-13	1
Karthik Raj	2025-05-13	1
Kiran	2025-05-08	1
Kiran	2025-05-21	2
Lakshmi Devi	2025-05-11	1
Manoj Jain	2025-05-18	1
Meena Patel	2025-05-14	1
Naveen Kumar	2025-05-16	1
Neha Reddy	2025-05-18	1
Pooja Gupta	2025-05-15	1
Priya	2025-05-03	1
Priya	2025-05-20	2

Patient_Name	Appointment_Date	Visit_Number
Rahul	2025-05-01	1
Rahul	2025-05-19	2
Ramesh Kumar	2025-05-10	1
Rohit Singh	2025-05-14	1
Shalini Verma	2025-05-17	1
Sneha	2025-05-06	1
Sneha	2025-05-21	2
Suresh Babu	2025-05-11	1
Vijay Sharma	2025-05-12	1

S7. Dense Ranking of Doctors by Revenue

Question: Assign dense rankings to doctors based on total billing revenue: Doctor Name, Revenue, Dense Rank.

Query:

```
-- derived table: a=doctors, b=appointments, c=billing
SELECT Doctor_Name, Revenue,
       DENSE_RANK() OVER (ORDER BY Revenue DESC) AS Dense_Rank_num
FROM (
    SELECT a.doctor_id, a.doctor_name AS Doctor_Name,
           SUM(c.bill_amount) AS Revenue
    FROM doctors a
    INNER JOIN appointments b
    ON b.doctor_id = a.doctor_id
    INNER JOIN billing c
    ON c.appointment_id = b.appointment_id
    GROUP BY a.doctor_id, a.doctor_name
) AS Doctor_revenue
ORDER BY Dense_Rank_num, Doctor_Name;
```

Output:

Doctor_Name	Revenue	Dense_Rank_num
-------------	---------	----------------

Doctor_Name	Revenue	Dense_Rank_num
Dr. Harish Rao	3600	1
Dr. Kavitha Devi	2800	2
Dr. Anjali Iyer	2700	3
Dr. Sunita Shah	2600	4
Dr. Rajesh Gupta	2550	5
Dr. Akash Verma	2400	6
Dr. Vivek Menon	2100	7
Dr. Kumar	2000	8
Dr. Mehta	1500	9
Dr. Sharma	1200	10
Dr. Prakash Nair	1100	11
Dr. Reddy	0	12

(No two doctors tie on revenue in this dataset, so DENSE_RANK produces the same sequence as RANK here.)

S8. Previous Appointment Date of Every Patient

Question: Display each appointment along with the previous appointment date of the same patient:
Patient Name, Current Appointment Date, Previous Appointment Date.

Query:

```
-- a=appointments, b=patients
SELECT b.patient_name AS Patient_Name, a.appointment_date AS Current_Appo
      LAG(a.appointment_date) OVER (
        PARTITION BY a.patient_id
        ORDER BY a.appointment_date, a.appointment_id
      ) AS Previous_Appointment_Date
FROM appointments a
INNER JOIN patients b
ON b.patient_id = a.patient_id
ORDER BY b.patient_name, a.appointment_date;
```

Output:

Patient_Name	Current_Appointment_Date	Previous_Appointment_Date
Ajay Das	2025-05-17	NULL
Anitha Rao	2025-05-12	NULL
Arun	2025-05-05	NULL
Arun	2025-05-20	2025-05-05
Deepa Nair	2025-05-13	NULL
Karthik Raj	2025-05-13	NULL
Kiran	2025-05-08	NULL
Kiran	2025-05-21	2025-05-08
Lakshmi Devi	2025-05-11	NULL
Manoj Jain	2025-05-18	NULL
Meena Patel	2025-05-14	NULL
Naveen Kumar	2025-05-16	NULL
Neha Reddy	2025-05-18	NULL
Pooja Gupta	2025-05-15	NULL
Priya	2025-05-03	NULL
Priya	2025-05-20	2025-05-03
Rahul	2025-05-01	NULL
Rahul	2025-05-19	2025-05-01
Ramesh Kumar	2025-05-10	NULL
Rohit Singh	2025-05-14	NULL
Shalini Verma	2025-05-17	NULL
Sneha	2025-05-06	NULL
Sneha	2025-05-21	2025-05-06
Suresh Babu	2025-05-11	NULL
Vijay Sharma	2025-05-12	NULL

S9. Next Appointment Date of Every Patient

Question: Display each appointment along with the next appointment date of the same patient: Patient Name, Current Appointment Date, Next Appointment Date.

Query:

```
-- a=appointments, b=patients
SELECT b.patient_name AS Patient_Name, a.appointment_date AS Current_Appointment_Date,
       LEAD(a.appointment_date) OVER (
         PARTITION BY a.patient_id
         ORDER BY a.appointment_date, a.appointment_id
       ) AS Next_Appointment_Date
FROM appointments a
INNER JOIN patients b
ON b.patient_id = a.patient_id
ORDER BY b.patient_name, a.appointment_date;
```

Output:

Patient_Name	Current_Appointment_Date	Next_Appointment_Date
Ajay Das	2025-05-17	NULL
Anitha Rao	2025-05-12	NULL
Arun	2025-05-05	2025-05-20
Arun	2025-05-20	NULL
Deepa Nair	2025-05-13	NULL
Karthik Raj	2025-05-13	NULL
Kiran	2025-05-08	2025-05-21
Kiran	2025-05-21	NULL
Lakshmi Devi	2025-05-11	NULL
Manoj Jain	2025-05-18	NULL
Meena Patel	2025-05-14	NULL
Naveen Kumar	2025-05-16	NULL
Neha Reddy	2025-05-18	NULL
Pooja Gupta	2025-05-15	NULL
Priya	2025-05-03	2025-05-20
Priya	2025-05-20	NULL

Patient_Name	Current_Appointment_Date	Next_Appointment_Date
Rahul	2025-05-01	2025-05-19
Rahul	2025-05-19	NULL
Ramesh Kumar	2025-05-10	NULL
Rohit Singh	2025-05-14	NULL
Shalini Verma	2025-05-17	NULL
Sneha	2025-05-06	2025-05-21
Sneha	2025-05-21	NULL
Suresh Babu	2025-05-11	NULL
Vijay Sharma	2025-05-12	NULL

S10. Highest Revenue Doctor in Each Specialization

Question: Identify the doctor who generated the highest billing revenue within each specialization:
Specialization, Doctor Name, Total Revenue.

Query:

```
-- derived table: a=doctors, b=appointments, c=billing
SELECT Specialization, Doctor_Name, Total_Revenue
FROM (
    SELECT a.specialization AS Specialization, a.doctor_name AS Doctor_Name,
           SUM(c.bill_amount) AS Total_Revenue,
           ROW_NUMBER() OVER (
               PARTITION BY a.specialization
               ORDER BY SUM(c.bill_amount) DESC, a.doctor_id
           ) AS Rn
    FROM doctors a
    INNER JOIN appointments b
    ON b.doctor_id = a.doctor_id
    INNER JOIN billing c
    ON c.appointment_id = b.appointment_id
    GROUP BY a.doctor_id, a.doctor_name, a.specialization
) AS Spec_revenue
WHERE Rn = 1
ORDER BY Specialization;
```

Output: (identical logic and result to M5 above – see M5 output table.)

S11. Rank Patients Based on Total Hospital Spending

Question: Rank patients according to total amount billed across all appointments: Patient Name, Total Billing Amount, Rank.

Query:

```
-- derived table: a=patients, b=appointments, c=billing
SELECT Patient_Name, Total_Billing_Amount,
       RANK() OVER (ORDER BY Total_Billing_Amount DESC) AS Rank_num
FROM (
  SELECT a.patient_id, a.patient_name AS Patient_Name,
         SUM(c.bill_amount) AS Total_Billing_Amount
  FROM patients a
  INNER JOIN appointments b
  ON b.patient_id = a.patient_id
  INNER JOIN billing c
  ON c.appointment_id = b.appointment_id
  GROUP BY a.patient_id, a.patient_name
) AS Patient_revenue
ORDER BY Rank_num, Patient_Name;
```

Output:

Patient_Name	Total_Billing_Amount	Rank_num
Priya	2400	1
Rahul	2100	2
Sneha	1900	3
Arun	1850	4
Deepa Nair	1800	5
Neha Reddy	1800	5
Karthik Raj	1400	7
Manoj Jain	1400	7
Anitha Rao	1300	9
Shalini Verma	1300	9

Patient_Name	Total_Billing_Amount	Rank_num
Ajay Das	1200	11
Vijay Sharma	1200	11
Ramesh Kumar	900	13
Rohit Singh	900	13
Lakshmi Devi	850	15
Pooja Gupta	850	15
Naveen Kumar	700	17
Suresh Babu	700	17
Kiran	0	19
Meena Patel	0	19

(Only 20 of the 20 patients have appointments/billing – all patients are represented.)

S12. Latest Treatment Received by Each Patient

Question: Display the most recent treatment received by every patient: Patient Name, Treatment Name, Appointment Date.

Query:

```
-- inner: a=patients, b=appointments, c=treatments
SELECT Patient_Name, Treatment_Name, Appointment_Date
FROM (
    SELECT a.patient_name AS Patient_Name, c.treatment_name AS Treatment_
           b.appointment_date AS Appointment_Date,
           ROW_NUMBER() OVER (
               PARTITION BY a.patient_id
               ORDER BY b.appointment_date DESC, b.appointment_id DESC
           ) AS Rn
    FROM patients a
    INNER JOIN appointments b
    ON b.patient_id = a.patient_id
    INNER JOIN treatments c
    ON c.appointment_id = b.appointment_id
) AS Ranked_treatments
```

```
WHERE Rn = 1
ORDER BY Patient_Name;
```

Output:

Patient_Name	Treatment_Name	Appointment_Date
Ajay Das	Stress Test	2025-05-17
Deepa Nair	MRI Scan	2025-05-13
Karthik Raj	Bone Scan	2025-05-13
Lakshmi Devi	Ear Cleaning	2025-05-11
Neha Reddy	Neurology Test	2025-05-18
Pooja Gupta	ENT Checkup	2025-05-15
Priya	Brain Scan	2025-05-03
Rahul	Pulmonary Function Test	2025-05-19
Ramesh Kumar	Vaccination	2025-05-10
Rohit Singh	Vaccination	2025-05-14
Shalini Verma	Pregnancy Scan	2025-05-17
Sneha	Knee X-Ray	2025-05-06
Suresh Babu	General Checkup	2025-05-11
Vijay Sharma	ECG	2025-05-12

(Only patients with at least one treatment record appear; 6 patients have none.)

S13. Top 2 Revenue-Generating Doctors

Question: Identify the top 2 doctors based on total revenue generated: Doctor Name, Total Revenue, Rank.

Query:

```
-- innermost derived table: a=doctors, b=appointments, c=billing
SELECT Doctor_Name, Total_Revenue, Rank_num
FROM (
    SELECT Doctor_Name, Total_Revenue,
           RANK() OVER (ORDER BY Total_Revenue DESC) AS Rank_num
```

```

FROM (
    SELECT a.doctor_id, a.doctor_name AS Doctor_Name,
           SUM(c.bill_amount) AS Total_Revenue
    FROM doctors a
    INNER JOIN appointments b
    ON b.doctor_id = a.doctor_id
    INNER JOIN billing c
    ON c.appointment_id = b.appointment_id
    GROUP BY a.doctor_id, a.doctor_name
) AS Doctor_revenue
) AS Ranked_revenue
WHERE Rank_num <= 2
ORDER BY Rank_num;

```

Output:

Doctor_Name	Total_Revenue	Rank_num
Dr. Harish Rao	3600	1
Dr. Kavitha Devi	2800	2

S14. Running Revenue Generated by Each Doctor

Question: Display the cumulative billing revenue generated by each doctor after every appointment:

Doctor Name, Appointment Date, Bill Amount, Running Revenue.

Query:

```

-- a=doctors, b=appointments, c=billing
SELECT a.doctor_name AS Doctor_Name, b.appointment_date AS Appointment_Date,
       SUM(c.bill_amount) OVER (
           PARTITION BY a.doctor_id
           ORDER BY b.appointment_date, b.appointment_id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS Running_Revenue
FROM doctors a
INNER JOIN appointments b
ON b.doctor_id = a.doctor_id
INNER JOIN billing c
ON c.appointment_id = b.appointment_id
ORDER BY a.doctor_name, b.appointment_date;

```

Output:

Doctor_Name	Appointment_Date	Bill_Amount	Running_Revenue
Dr. Akash Verma	2025-05-12	1200	1200
Dr. Akash Verma	2025-05-17	1200	2400
Dr. Anjali Iyer	2025-05-10	900	900
Dr. Anjali Iyer	2025-05-14	900	1800
Dr. Anjali Iyer	2025-05-20	900	2700
Dr. Harish Rao	2025-05-13	1800	1800
Dr. Harish Rao	2025-05-18	1800	3600
Dr. Kavitha Devi	2025-05-13	1400	1400
Dr. Kavitha Devi	2025-05-18	1400	2800
Dr. Kumar	2025-05-01	1000	1000
Dr. Kumar	2025-05-05	1000	2000
Dr. Mehta	2025-05-03	1500	1500
Dr. Prakash Nair	2025-05-14	0	0
Dr. Prakash Nair	2025-05-19	1100	1100
Dr. Rajesh Gupta	2025-05-11	850	850
Dr. Rajesh Gupta	2025-05-15	850	1700
Dr. Rajesh Gupta	2025-05-20	850	2550
Dr. Reddy	2025-05-08	0	0
Dr. Sharma	2025-05-06	1200	1200
Dr. Sunita Shah	2025-05-12	1300	1300
Dr. Sunita Shah	2025-05-17	1300	2600
Dr. Sunita Shah	2025-05-21	0	2600
Dr. Vivek Menon	2025-05-11	700	700
Dr. Vivek Menon	2025-05-16	700	1400
Dr. Vivek Menon	2025-05-21	700	2100

S15. Difference Between Current and Previous Bill

Question: Display the difference between current and previous bill amount for every patient: Patient Name, Appointment Date, Current Bill, Previous Bill, Difference.

Query:

```
-- a=patients, b=appointments, c=billing
SELECT a.patient_name AS Patient_Name, b.appointment_date AS Appointment_
      LAG(c.bill_amount) OVER (
        PARTITION BY a.patient_id
        ORDER BY b.appointment_date, b.appointment_id
      ) AS Previous_Bill,
      c.bill_amount - LAG(c.bill_amount) OVER (
        PARTITION BY a.patient_id
        ORDER BY b.appointment_date, b.appointment_id
      ) AS Difference
FROM patients a
INNER JOIN appointments b
ON b.patient_id = a.patient_id
INNER JOIN billing c
ON c.appointment_id = b.appointment_id
ORDER BY a.patient_name, b.appointment_date;
```

Output:

Patient_Name	Appointment_Date	Current_Bill	Previous_Bill	Difference
Ajay Das	2025-05-17	1200	NULL	NULL
Anitha Rao	2025-05-12	1300	NULL	NULL
Arun	2025-05-05	1000	NULL	NULL
Arun	2025-05-20	850	1000	-150
Deepa Nair	2025-05-13	1800	NULL	NULL
Karthik Raj	2025-05-13	1400	NULL	NULL
Kiran	2025-05-08	0	NULL	NULL
Kiran	2025-05-21	0	0	0
Lakshmi Devi	2025-05-11	850	NULL	NULL
Manoj Jain	2025-05-18	1400	NULL	NULL
Meena Patel	2025-05-14	0	NULL	NULL
Naveen Kumar	2025-05-16	700	NULL	NULL

Patient_Name	Appointment_Date	Current_Bill	Previous_Bill	Difference
Neha Reddy	2025-05-18	1800	NULL	NULL
Pooja Gupta	2025-05-15	850	NULL	NULL
Priya	2025-05-03	1500	NULL	NULL
Priya	2025-05-20	900	1500	-600
Rahul	2025-05-01	1000	NULL	NULL
Rahul	2025-05-19	1100	1000	100
Ramesh Kumar	2025-05-10	900	NULL	NULL
Rohit Singh	2025-05-14	900	NULL	NULL
Shalini Verma	2025-05-17	1300	NULL	NULL
Sneha	2025-05-06	1200	NULL	NULL
Sneha	2025-05-21	700	1200	-500
Suresh Babu	2025-05-11	700	NULL	NULL
Vijay Sharma	2025-05-12	1200	NULL	NULL

S16. Most Frequently Visited Doctor for Each Patient

Question: Identify the doctor whom each patient has visited the most: Patient Name, Doctor Name, Total Visits. One doctor per patient.

Query:

```
-- a=patients, b=appointments, c=doctors
SELECT Patient_Name, Doctor_Name, Total_Visits
FROM (
    SELECT a.patient_name AS Patient_Name, c.doctor_name AS Doctor_Name,
           COUNT(*) AS Total_Visits,
           ROW_NUMBER() OVER (
               PARTITION BY a.patient_id
               ORDER BY COUNT(*) DESC, c.doctor_id ASC
           ) AS Rn
    FROM patients a
    INNER JOIN appointments b
    ON b.patient_id = a.patient_id
    INNER JOIN doctors c
```

```

ON c.doctor_id = b.doctor_id
GROUP BY a.patient_id, a.patient_name, c.doctor_id, c.doctor_name
) AS Ranked_visits
WHERE Rn = 1
ORDER BY Patient_Name;

```

Output:

Patient_Name	Doctor_Name	Total_Visits
Ajay Das	Dr. Akash Verma	1
Anitha Rao	Dr. Sunita Shah	1
Arun	Dr. Kumar	1
Deepa Nair	Dr. Harish Rao	1
Karthik Raj	Dr. Kavitha Devi	1
Kiran	Dr. Reddy	1
Lakshmi Devi	Dr. Rajesh Gupta	1
Manoj Jain	Dr. Kavitha Devi	1
Meena Patel	Dr. Prakash Nair	1
Naveen Kumar	Dr. Vivek Menon	1
Neha Reddy	Dr. Harish Rao	1
Pooja Gupta	Dr. Rajesh Gupta	1
Priya	Dr. Mehta	1
Rahul	Dr. Kumar	1
Ramesh Kumar	Dr. Anjali Iyer	1
Rohit Singh	Dr. Anjali Iyer	1
Shalini Verma	Dr. Sunita Shah	1
Sneha	Dr. Sharma	1
Suresh Babu	Dr. Vivek Menon	1
Vijay Sharma	Dr. Akash Verma	1

(Every patient in this dataset has at most one visit per doctor, so all counts are 1. For patients with two appointments to two different doctors – Rahul, Priya, Sneha, Kiran, Arun – the tie is broken by the lower doctor_id.)

S17. Rank Treatments by Cost Within Each Doctor

Question: Rank treatments handled by each doctor based on treatment cost: Doctor Name, Treatment Name, Treatment Cost, Treatment Rank.

Query:

```
-- a=doctors, b=appointments, c=treatments
SELECT a.doctor_name AS Doctor_Name, c.treatment_name AS Treatment_Name,
       RANK() OVER (
           PARTITION BY a.doctor_id
           ORDER BY c.treatment_cost DESC
       ) AS Treatment_Rank
FROM doctors a
INNER JOIN appointments b
ON b.doctor_id = a.doctor_id
INNER JOIN treatments c
ON c.appointment_id = b.appointment_id
ORDER BY a.doctor_name, Treatment_Rank;
```

Output:

Doctor_Name	Treatment_Name	Treatment_Cost	Treatment_Rank
Dr. Akash Verma	Stress Test	900	1
Dr. Akash Verma	ECG	500	2
Dr. Anjali Iyer	Vaccination	300	1
Dr. Anjali Iyer	Vaccination	300	1
Dr. Harish Rao	Neurology Test	4000	1
Dr. Harish Rao	MRI Scan	3500	2
Dr. Kavitha Devi	Bone Scan	1800	1
Dr. Kumar	ECG	500	1
Dr. Mehta	Brain Scan	2500	1
Dr. Prakash Nair	Pulmonary Function Test	1600	1
Dr. Rajesh Gupta	ENT Checkup	500	1
Dr. Rajesh Gupta	Ear Cleaning	400	2
Dr. Sharma	Knee X-Ray	1200	1

Doctor_Name	Treatment_Name	Treatment_Cost	Treatment_Rank
Dr. Sunita Shah	Pregnancy Scan	2200	1
Dr. Vivek Menon	General Checkup	250	1

(Doctors with only one treatment naturally rank 1. Dr. Reddy has no treatments and is excluded by the INNER JOIN.)

S18. First Appointment of Every Patient

Question: Display the first appointment recorded for every patient: Patient Name, Appointment Date, Doctor Name.

Query:

```
-- inner: a=appointments, b=patients, c=doctors
SELECT Patient_Name, Appointment_Date, Doctor_Name
FROM (
    SELECT b.patient_name AS Patient_Name, a.appointment_date AS Appointn
        c.doctor_name AS Doctor_Name,
        ROW_NUMBER() OVER (
            PARTITION BY a.patient_id
            ORDER BY a.appointment_date ASC, a.appointment_id ASC
        ) AS Rn
    FROM appointments a
    INNER JOIN patients b
    ON b.patient_id = a.patient_id
    INNER JOIN doctors c
    ON c.doctor_id = a.doctor_id
) AS Ranked_appointments
WHERE Rn = 1
ORDER BY Patient_Name;
```

Output:

Patient_Name	Appointment_Date	Doctor_Name
Ajay Das	2025-05-17	Dr. Akash Verma
Anitha Rao	2025-05-12	Dr. Sunita Shah
Arun	2025-05-05	Dr. Kumar

Patient_Name	Appointment_Date	Doctor_Name
Deepa Nair	2025-05-13	Dr. Harish Rao
Karthik Raj	2025-05-13	Dr. Kavitha Devi
Kiran	2025-05-08	Dr. Reddy
Lakshmi Devi	2025-05-11	Dr. Rajesh Gupta
Manoj Jain	2025-05-18	Dr. Kavitha Devi
Meena Patel	2025-05-14	Dr. Prakash Nair
Naveen Kumar	2025-05-16	Dr. Vivek Menon
Neha Reddy	2025-05-18	Dr. Harish Rao
Pooja Gupta	2025-05-15	Dr. Rajesh Gupta
Priya	2025-05-03	Dr. Mehta
Rahul	2025-05-01	Dr. Kumar
Ramesh Kumar	2025-05-10	Dr. Anjali Iyer
Rohit Singh	2025-05-14	Dr. Anjali Iyer
Shalini Verma	2025-05-17	Dr. Sunita Shah
Sneha	2025-05-06	Dr. Sharma
Suresh Babu	2025-05-11	Dr. Vivek Menon
Vijay Sharma	2025-05-12	Dr. Akash Verma

S19. Doctor Revenue Contribution Percentage

Question: Calculate each doctor's percentage contribution to the hospital's total billing revenue: Doctor Name, Revenue Generated, Revenue Contribution %.

Query:

```
-- a=doctors, b=appointments, c=billing
SELECT a.doctor_name AS Doctor_Name,
       SUM(c.bill_amount) AS Revenue_Generated,
       ROUND(
         100.0 * SUM(c.bill_amount) / (SELECT SUM(bill_amount) FROM bil
         2
       ) AS Revenue_Contribution_Pct
```

```

FROM doctors a
INNER JOIN appointments b
ON b.doctor_id = a.doctor_id
INNER JOIN billing c
ON c.appointment_id = b.appointment_id
GROUP BY a.doctor_id, a.doctor_name
ORDER BY Revenue_Contribution_Pct DESC;

```

Output:

Doctor_Name	Revenue_Generated	Revenue_Contribution_Pct
Dr. Harish Rao	3600	14.66
Dr. Kavitha Devi	2800	11.41
Dr. Anjali Iyer	2700	11.00
Dr. Sunita Shah	2600	10.59
Dr. Rajesh Gupta	2550	10.39
Dr. Akash Verma	2400	9.78
Dr. Vivek Menon	2100	8.55
Dr. Kumar	2000	8.15
Dr. Mehta	1500	6.11
Dr. Sharma	1200	4.89
Dr. Prakash Nair	1100	4.48
Dr. Reddy	0	0.00

(Total hospital billing revenue = 24550.)

S20. Top 3 Patients by Spending in Each City

Question: Find the top 3 patients based on total billing amount within each city: City, Patient Name, Total Billing Amount, Rank within City.

Query:

```

-- derived table: a=patients, b=appointments, c=billing
SELECT City, Patient_Name, Total_Billing_Amount, Rank_within_city
FROM (

```

```

SELECT a.city AS City, a.patient_name AS Patient_Name,
       SUM(c.bill_amount) AS Total_Billing_Amount,
       RANK() OVER (
           PARTITION BY a.city
           ORDER BY SUM(c.bill_amount) DESC
       ) AS Rank_within_city
FROM patients a
INNER JOIN appointments b
ON b.patient_id = a.patient_id
INNER JOIN billing c
ON c.appointment_id = b.appointment_id
GROUP BY a.patient_id, a.patient_name, a.city
) AS City_ranked
WHERE Rank_within_city <= 3
ORDER BY City, Rank_within_city;

```

Output:

City	Patient_Name	Total_Billing_Amount	Rank_within_city
Ahmedabad	Meena Patel	0	1
Bangalore	Priya	2400	1
Bangalore	Anitha Rao	1300	2
Chennai	Rahul	2100	1
Chennai	Karthik Raj	1400	2
Chennai	Ramesh Kumar	900	3
Coimbatore	Lakshmi Devi	850	1
Delhi	Rohit Singh	900	1
Delhi	Kiran	0	2
Hyderabad	Arun	1850	1
Hyderabad	Neha Reddy	1800	2
Hyderabad	Pooja Gupta	850	3
Jaipur	Manoj Jain	1400	1
Kochi	Deepa Nair	1800	1
Kolkata	Ajay Das	1200	1
Lucknow	Shalini Verma	1300	1

City	Patient_Name	Total_Billing_Amount	Rank_within_city
Madurai	Suresh Babu	700	1
Mumbai	Sneha	1900	1
Mumbai	Vijay Sharma	1200	2
Pune	Naveen Kumar	700	1

(Cities with only one patient in this dataset – Ahmedabad, Coimbatore, Jaipur, Kochi, Kolkata, Lucknow, Madurai, Pune – naturally show only 1 row since there's nothing to rank against.)